



Departamento de Electrónica, Sistemas e Informática

Big Data

Spring 2026

Examples on Machine Learning: Logistic Regression

Profesor: Pablo Camarillo Ramirez

Alumno: Bryan Edgardo Romo Gonzalez

Create SparkSession

```
In [1]: from pcamarillor.spark_utils import SparkUtils
su = SparkUtils("ML: Logistic Regression",
                "spark://spark-master:7077")
su.spark
```

```
WARNING: Using incubator modules: jdk.incubator.vector
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
26/04/17 03:17:11 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
```

Out[1]: **SparkSession - in-memory**

SparkContext

Spark UI

Version	v4.0.1
Master	spark://spark-master:7077
AppName	ML: Logistic Regression

Collect Data

```
In [3]: from pcamarillor.spark_utils import SparkUtils
# Create a small dataset as a list of tuples
# Format: (label, feature_x1, feature_x2)
data = [
    (1.0, 2.0, 3.0),
    (0.0, 1.0, 2.5),
    (1.0, 3.0, 5.0),
    (0.0, 0.5, 1.0),
    (1.0, 4.0, 6.0)
]

# Define schema for the DataFrame
schema = SparkUtils.generate_schema([("label", "float"),
                                     ("feature_x1", "float"),
                                     ("feature_x2", "float")])

# Convert list to a DataFrame
df = su.spark.createDataFrame(data, schema)
```

Assemble the features into a single vector column

```
In [5]: from pyspark.ml.feature import VectorAssembler

assembler = VectorAssembler(inputCols=["feature_x1", "feature_x2"], outputCol="features")
data_with_features = assembler.transform(df).select("label", "features")
data_with_features.show()
data_with_features.printSchema()
```

[Stage 0:> (0 + 1) / 1]

```
+-----+-----+
|label| features|
+-----+-----+
|  1.0|[2.0,3.0]|
|  0.0|[1.0,2.5]|
|  1.0|[3.0,5.0]|
|  0.0|[0.5,1.0]|
|  1.0|[4.0,6.0]|
+-----+-----+
```

```
root
|-- label: float (nullable = true)
|-- features: vector (nullable = true)
```

Data splitting

80% training data and 20% testing data

```
In [6]: train_df, test_df = data_with_features.randomSplit([0.8, 0.2], seed=57)
```

Show dataset (for debugging)

```
In [9]: print("Original Dataset")
df.show()

# Print train dataset
print("train set")
train_df.show()
```

Original Dataset

```
+-----+-----+-----+
|label|feature_x1|feature_x2|
+-----+-----+-----+
| 1.0|      2.0|      3.0|
| 0.0|      1.0|      2.5|
| 1.0|      3.0|      5.0|
| 0.0|      0.5|      1.0|
| 1.0|      4.0|      6.0|
+-----+-----+-----+
```

train set

```
+-----+-----+
|label| features|
+-----+-----+
| 0.0|[1.0,2.5]|
| 1.0|[2.0,3.0]|
| 0.0|[0.5,1.0]|
| 1.0|[4.0,6.0]|
+-----+-----+
```

Create ML Model

```
In [7]: from pyspark.ml.classification import LogisticRegression
lr = LogisticRegression(maxIter=10, regParam=0.01)
```

Train ML Model

```
In [8]: lr_model = lr.fit(train_df)

# Print coefficients
print("Coefficients: " + str(lr_model.coefficients))

# Display model summary
training_summary = lr_model.summary
```

26/04/16 03:11:15 WARN InstanceBuilder: Failed to load implementation from:dev.ludovic.netlib.blas.JNIBLAS
Coefficients: [2.346116998875653,0.7963873036415706]

Predictions

```
In [10]: # Use the trained model to make predictions on the test data
predictions = lr_model.transform(test_df)

# Show predictions
predictions.select("features", "prediction", "probability").show()
```

```
+-----+-----+-----+
| features|prediction|      probability|
+-----+-----+-----+
|[3.0,5.0]|      1.0|[0.00524886113385...|
+-----+-----+-----+
```

Test ML Model

```
In [11]: from pyspark.ml.evaluation import MulticlassClassificationEvaluator

evaluator = MulticlassClassificationEvaluator(labelCol="label",
                                              predictionCol="prediction")

accuracy = evaluator.evaluate(predictions,
                             {evaluator.metricName: "accuracy"})
print(f"Accuracy: {accuracy}")
precision = evaluator.evaluate(predictions,
```

```
        {evaluator.metricName: "weightedPrecision"})
print(f"Precision: {precision}")
recall = evaluator.evaluate(predictions,
        {evaluator.metricName: "weightedRecall"})
print(f"Recall: {recall}")
f1 = evaluator.evaluate(predictions,
        {evaluator.metricName: "f1"})
print(f"F1 Score: {f1}")
```

Accuracy: 1.0
Precision: 1.0
Recall: 1.0
F1 Score: 1.0

Lab 10: Logistic regression to predict heart disease

Data collection

```
In [2]: # Define schema for the DataFrame
heart_schema = SparkUtils.generate_schema([
    ("male", "int"),
    ("age", "int"),
    ("education", "int"),
    ("currentSmoker", "int"),
    ("cigsPerDay", "int"),
    ("BPMeds", "int"),
    ("prevalentStroke", "int"),
    ("prevalentHyp", "int"),
    ("diabetes", "int"),
    ("totChol", "int"),
    ("sysBP", "float"),
    ("diaBP", "float"),
    ("BMI", "float"),
    ("heartRate", "int"),
    ("glucose", "int"),
    ("TenYearCHD", "int")]) # columna que dice si el paciente tuvo un ataque al corazon
```

```
# Source: https://www.kaggle.com/dileep070/heart-disease-prediction-using-logistic-regression?resource=download

heart_df = su.spark.read \
    .option("header", "true") \
    .schema(heart_schema) \
    .csv("/opt/spark/work-dir/data/ml/logistic_regression/framingham.csv")

heart_df.printSchema()
```

```
root
|-- male: integer (nullable = true)
|-- age: integer (nullable = true)
|-- education: integer (nullable = true)
|-- currentSmoker: integer (nullable = true)
|-- cigsPerDay: integer (nullable = true)
|-- BPMeds: integer (nullable = true)
|-- prevalentStroke: integer (nullable = true)
|-- prevalentHyp: integer (nullable = true)
|-- diabetes: integer (nullable = true)
|-- totChol: integer (nullable = true)
|-- sysBP: float (nullable = true)
|-- diaBP: float (nullable = true)
|-- BMI: float (nullable = true)
|-- heartRate: integer (nullable = true)
|-- glucose: integer (nullable = true)
|-- TenYearCHD: integer (nullable = true)
```

```
In [3]: heart_df.show(2)
```

```
[Stage 0:>
```

```
(0 + 1) / 1]
```

```

+---+---+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
-----+-----+
|male|age|education|currentSmoker|cigsPerDay|BPMeds|prevalentStroke|prevalentHyp|diabetes|totChol|sysBP|diaBP| BMI|heartRate|g
lucose|TenYearCHD|
+---+---+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
-----+-----+
|  1| 39|      4|      0|      0|      0|      0|      0|      0|    195|106.0| 70.0|26.97|      80|
77|      0|
|  0| 46|      2|      0|      0|      0|      0|      0|      0|    250|121.0| 81.0|28.73|      95|
76|      0|
+---+---+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
-----+-----+
only showing top 2 rows

```

```

In [4]: from pyspark.ml.feature import VectorAssembler

# eliminat nulls
clean_df = heart_df.dropna()

assembler = VectorAssembler(
    inputCols=[
        "male", "age", "education", "currentSmoker", "cigsPerDay",
        "BPMeds", "prevalentStroke", "prevalentHyp", "diabetes",
        "totChol", "sysBP", "diaBP", "BMI", "heartRate", "glucose"
    ],
    outputCol="features"
)

heart_features = assembler.transform(clean_df) \
    .withColumnRenamed("TenYearCHD", "label") \
    .select("label", "features")

heart_features.show(5)

```

26/04/17 03:17:33 WARN SparkStringUtils: Truncated the string representation of a plan since it was too large. This behavior can be adjusted by setting 'spark.sql.debug.maxToStringFields'.

[Stage 1:> (0 + 1) / 1]

```
+-----+-----+
|label|          features|
+-----+-----+
|    0|[1.0,39.0,4.0,0.0...|
|    0|(15,[1,2,9,10,11,...|
|    0|[1.0,48.0,1.0,1.0...|
|    1|[0.0,61.0,3.0,1.0...|
|    0|[0.0,46.0,3.0,1.0...|
+-----+-----+
only showing top 5 rows
```

Data Splitting

```
In [5]: train_df, test_df = heart_features.randomSplit([0.8, 0.2], seed=57)
```

```
In [6]: heart_features.show(5)
```

```
+-----+-----+
|label|          features|
+-----+-----+
|    0|[1.0,39.0,4.0,0.0...|
|    0|(15,[1,2,9,10,11,...|
|    0|[1.0,48.0,1.0,1.0...|
|    1|[0.0,61.0,3.0,1.0...|
|    0|[0.0,46.0,3.0,1.0...|
+-----+-----+
only showing top 5 rows
```

```
In [7]: print("Train set")
        train_df.show(5)
```

Train set

```

+-----+-----+
|label|          features|
+-----+-----+
|    0|(15,[1,2,9,10,11,...|
|    0|(15,[1,2,9,10,11,...|
|    0|(15,[1,2,9,10,11,...|
|    0|(15,[1,2,9,10,11,...|
|    0|(15,[1,2,9,10,11,...|
+-----+-----+

```

only showing top 5 rows

Create ML Model

```
In [8]: from pyspark.ml.classification import LogisticRegression
```

```
lr = LogisticRegression(maxIter=10, regParam=0.01)
```

Train ML Model

```
In [9]: lr_model = lr.fit(train_df)
```

```

# Coeficientes del modelo
print("Coefficients:", lr_model.coefficients)

```

```

# Resumen del entrenamiento
training_summary = lr_model.summary

```

26/04/17 03:18:05 WARN InstanceBuilder: Failed to load implementation from:dev.ludovic.netlib.blas.JNIBLAS

```

Coefficients: [0.560736387880166,0.058016343671351826,-0.03862329388060508,0.08452119504869099,0.01367774812800251,0.2078776671
691058,0.6011410128804235,0.23123946470587214,0.1554414002615105,0.0018853859746570058,0.013440763204978253,0.00041310789946948
583,0.005988278680733878,-0.0016384244772548002,0.007162714926668017]

```

```
In [10]: # Usar el modelo para predecir
predictions = lr_model.transform(test_df)
```

```
# Mostrar resultados
predictions.select("label", "prediction", "probability").show(10)
```

[Stage 17:> (0 + 1) / 1]

```
+-----+-----+-----+
|label|prediction|probability|
+-----+-----+-----+
|  0  |      0.0|[0.97710931659216...|
|  0  |      0.0|[0.97113718941976...|
|  0  |      0.0|[0.96821335313213...|
|  0  |      0.0|[0.98006272991873...|
|  0  |      0.0|[0.95368213163393...|
|  0  |      0.0|[0.96837933407918...|
|  0  |      0.0|[0.97360711575497...|
|  0  |      0.0|[0.96998684843853...|
|  0  |      0.0|[0.95453168435392...|
|  0  |      0.0|[0.96422850098125...|
+-----+-----+-----+
```

only showing top 10 rows

Test ML Model

```
In [11]: from pyspark.ml.evaluation import MulticlassClassificationEvaluator
```

```
evaluator = MulticlassClassificationEvaluator(
    labelCol="label",
    predictionCol="prediction"
)
```

```
accuracy = evaluator.evaluate(predictions, {
    evaluator.metricName: "accuracy"
})
print(f"Accuracy: {accuracy}")
```

```
precision = evaluator.evaluate(predictions, {
    evaluator.metricName: "weightedPrecision"
```

```
})  
print(f"Precision: {precision}")  
  
recall = evaluator.evaluate(predictions, {  
    evaluator.metricName: "weightedRecall"  
})  
print(f"Recall: {recall}")  
  
f1 = evaluator.evaluate(predictions, {  
    evaluator.metricName: "f1"  
})  
print(f"F1 Score: {f1}")
```

```
Accuracy: 0.8299866131191432  
Precision: 0.8038821954484605  
Recall: 0.8299866131191433  
F1 Score: 0.7697118793492835
```

```
In [ ]: su.spark.stop()
```